

Objectives

After completing this lab, I will be able to do the following:

- Review an Amazon Elastic Compute Cloud (Amazon EC2) Instance and web application and validate the configuration.
- Create an Application Load Balancer and launch template.
- Set up an Amazon EC2 Auto Scaling group.
- Launch a template.
- Stress test a web application to validate scaling.

Scenario

The new Employee Directory application was a huge success. The HR team was thrilled! My manager is concerned that the application will be so popular that there could be performance issues. To get ahead of this problem, she tasks me with setting up an Application Load Balancer and Auto Scaling group to allow the application to scale horizontally with the demand of the company.

In this lab, I'll create a target group, an Application Load Balancer, and a launch template. Then, I'll set up an Auto Scaling group that uses the target group, Application Load Balancer and the launch template I created. Finally, I'll test the application with a simulated work load and watch my application scale in real time.

Task 1: Review an EC2 Instance and web application to validate the configuration

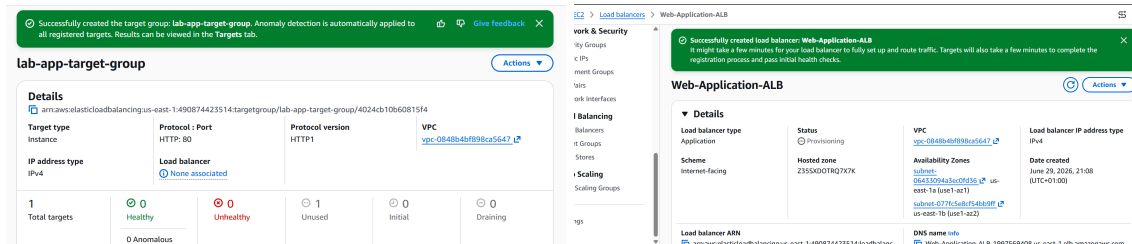
The screenshot displays the AWS Management Console interface. On the left, the navigation pane shows the 'Instances' section selected. The main content area is divided into two panels. The left panel, titled 'Instance summary for i-07e2ecf3124b2e7e1 (Web Application)', provides details for the EC2 instance, including its ID, IP addresses, hostname type, and VPC ID. The right panel, titled 'Employee Directory - Configuration', shows the configuration settings for the application, including settings for DynamoDB, S3, and the EC2 instance itself.

Setting	Value
DynamoDB Enabled	☒
S3 Access Enabled	☒
S3 Bucket	labstack-de022752-3a05-4ede-a544-0d71-imagebucket-kimthibkwohm
Region	us-east-1
Availability Zone	us-east-1a
EC2 Instance Id	i-07e2ecf3124b2e7e1

I successfully reviewed the EC2 Instance and Employee Directory application.

Task 2: Create an Application Load Balancer

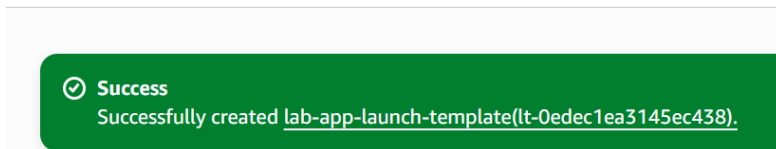
In this task, I'll create an Application Load Balancer, a required target group, and a Launch Template.



I successfully created an Application Load Balancer, target group, security group and confirmed the Employee Directory application has been set up correctly.

Task 3: Create a Launch Template

Now that I can access my application from a single DNS name, I can scale horizontally. To do so, I need an Auto Scaling group. But before I can create an Auto Scaling group, I need a launch template. In this task, I'll create a launch template to use later when you create an Auto Scaling group.



This script below performs the following tasks:

- Installs system updates
- Installs a source repository so the Node.js installer can be downloaded
- Installs Node.js
- Installs the EPEL repository that provides additional options for package installation
- Installs the stress package to stress the CPU
- Downloads the application code
- Creates a dedicated directory for the web application
- Downloads and deploys (extracts) the application into the specified directory
- Tells the application the bucket where the images are available
- Tells the application the Region where the application is running
- Installs the application dependencies
- Starts the web application

```

#!/bin/bash -ex

# Update these three variables with values to the left of these instructions

IMAGES_BUCKET=UPDATE_WITH_IMAGES_BUCKET_NAME

INSTALLATION_BUCKET=UPDATE_WITH_INSTALLATION_BUCKET_NAME

YOUR_DEFAULT_AWS_REGION=UPDATE_WITH_ACTUAL_REGION

# Update yum

yum -y update

# installs nvm (Node Version Manager)

#curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.0/install.sh | bash

# Download and install nvm

aws s3 cp s3://$INSTALLATION_BUCKET/install.sh - | bash

export NVM_DIR="$HOME/.nvm"

[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm

[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" # This loads nvm bash_completion

# download and install Node.js (you may need to restart the terminal)

nvm install 20

#install stress tool (for load balancing testing)

yum -y install stress

# Create a dedicated directory for the application

mkdir -p /var/app

# Get the app from Amazon S3

#wget https://aws-tc-largeobjects.s3-us-west-2.amazonaws.com/ILT-TF-100-TECESS-5/app/app.zip

# Download app from S3

aws s3 cp s3://$INSTALLATION_BUCKET/app.zip .

# Extract it into a desired folder

unzip app.zip -d /var/app/

cd /var/app/

# Configure S3 bucket details

export PHOTOS_BUCKET=$IMAGES_BUCKET

# Configure default AWS Region

export DEFAULT_AWS_REGION=$YOUR_DEFAULT_AWS_REGION

# Enable admin tools for stress testing

export SHOW_ADMIN_TOOLS=1

# Install dependencies

npm install

# Start your app

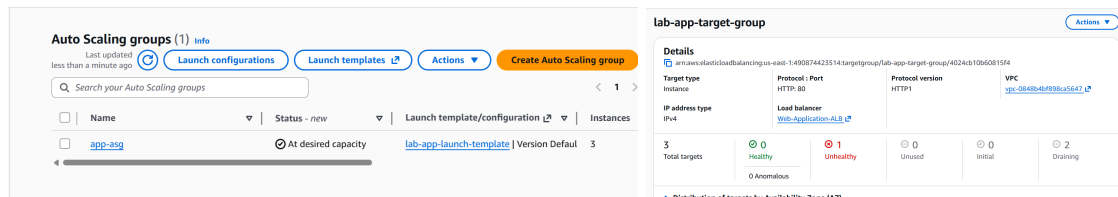
npm start

```

I successfully created a Launch Template.

Task 4: Create an Auto Scaling group

In this task, I'll use my Launch Template to create an Auto Scaling group.



I successfully created an Auto Scaling group using a launch template and verified the addition of new EC2 instances to the target group.

Task 5: Test the Application

In this task, I'll test the employee directory application and validate the application is load balancing between different availability zones. Then, I'll conduct a load test to trigger a scaling event to test the horizontal scaling of my application.

Lab Complete

I can now:

- Review an EC2 Instance and Web Application to Validate the Configuration
- Create an Application Load Balancer and Launch Template
- Set up an Amazon Elastic Compute Cloud (Amazon EC2) Auto Scaling group
- Launch a template
- Stress test a web application to validate scaling

✔ Successfully completed lab.